

Step 1: Set Up Development Environment

1. Install Node.js and npm.
2. Set up a new Hardhat project:
 - Create a new directory for your project and navigate into it.
 - Run `npm init -y` followed by `npm install --save-dev hardhat`.
3. Install OpenZeppelin contracts:

bash

```
npm install @openzeppelin/contracts
```

Step 2: Write the ERC20 Token Contract

1. Navigate to the `contracts` directory in your Hardhat project.
2. Create a new file named `MyToken.sol`.
3. Implement the ERC20 token using OpenZeppelin's library:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
contract MyToken is ERC20 {
  constructor(uint256 initialSupply) ERC20("MyToken", "MTK") {
    _mint(msg.sender, initialSupply);
  }
}
```

4. Save the file.

Step 3: Write a Deployment Script

1. Navigate to the `scripts` directory.
2. Create a new file named `deploy.js`.
3. Add the following code to deploy the contract:

```
const { ethers } = require("hardhat");
async function main() {
  const [deployer] = await ethers.getSigners();
  console.log("Deploying contract with account:", deployer.address);
  const MyToken = await ethers.getContractFactory("MyToken");
  const myToken = await MyToken.deploy(ethers.utils.parseEther("1000000"));
  await myToken.deployed();
  console.log("MyToken deployed to:", myToken.address);
}
main().catch((error) => { console.error(error); process.exitCode = 1; });
```

Step 4: Compile and Deploy

1. Compile the contract:

bash

```
npx hardhat compile
```

2. Deploy it to a local Hardhat network:

bash

```
npx hardhat run scripts/deploy.js --network localhost
```

Step 5: Test Your Token

1. Write tests in Hardhat using Mocha and Chai or use Foundry for Solidity-based testing.
2. Example test (using Hardhat):

javascript

```
const { expect } = require("chai"); describe("MyToken", function () { it("Should deploy with correct initial supply", async function () { const [deployer] = await ethers.getSigners(); const MyToken = await ethers.getContractFactory("MyToken"); const myToken = await MyToken.deploy(ethers.utils.parseEther("1000000")); expect(await myToken.balanceOf(deployer.address)).to.equal(ethers.utils.parseEther("1000000")); }); });
```

Step 6: Extend Functionality

- Add a minting function restricted to the owner.
- Implement token transfers between accounts.
- Test edge cases like transferring more tokens than available balance.

Deliverables

- The Solidity contract (**MyToken.sol**).
- Deployment script (**deploy.js**).
- Test cases for minting, transferring, and balance checks.